

GamutX

Absolute & Relatives Paths

Python Basics

File Handling by Absolute & Relative Path

- What Is a Path?
- Why Understanding Paths Matters
- How Do We Solve This?

What Is a Path ?

- A **path** tells Python **where a file or folder is on your computer.**
- Think of it like giving someone directions to a specific file.
- Whenever we want to:
 - **Open** a texture file
 - **Read** a data config
 - **Write** output or logs
- **we have to tell Python exactly where that file is**

Types of Paths

1- Absolute Path

- Specifies the **full location** of a file.
- Works reliably, **but isn't portable across machines.**

- **Example:**

- **Windows** → `C:\Users\John\Documents\project\data.txt`

- **macOS/Linux** → `/home/john/project/data.txt`

2- Relative Path

- Describes how to get to the **file starting** from the **current working directory.**
- Works better for projects that **need to run on different systems** or for **teams sharing code.**

- **Example:**

- **Windows** → `"data\data.txt"`

- **macOS/Linux** → `"data/data.txt"`

Relative Path

- What is a Relative Path?
- File in the Same Directory (Root)
- File in a Subfolder
- File in Multiple Subfolders
- A Large Project Structure

What are Relative Path ?

A Relative path is a way to tell Python where a file is located **based on the current working directory (CWD)**.

What Is the "Current Working Directory" (CWD)?

- The **current working directory (CWD)** is the folder from which we run the script.

Let's say your folder looks like this:

```
test/├── project/│   ├── data/│       └── sample.txt│   └── script.py
```

CWD

current working directory

" CWD = Our Project Root Folder "

Terminal

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pralhad\Desktop\project>
└──┬──┘
   CWD
```

File in the Same Directory (Root)

What if the file and the Python script are in the same folder?

Folder Structure:

```
repo/  
├── sample.txt  
└── main.py
```

Content of sample.txt:

```
This is a test line.
```

Code in main.py:

```
with open("sample.txt", "r") as f:  
    content = f.read()  
  
print(content)  
# Output : This is a test line.
```

CWD / Project Root

Since the file is in the same folder as main.py, we can simply use the filename.

File in a Subfolder

Now let's move the file into a subfolder.

Folder Structure:

```
repo/  
├── data/  
│   └── sample.txt  
└── main.py
```

Here:

- **main.py** is in **project folder**
- **sample.txt** → moved into the **data/** subfolder.

What Happens When We Run the Script?

```
with open("sample.txt", "r") as f:  
    content = f.read()  
  
print(content)
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS C:\Users\pralhad\Desktop\ma> & C:/Users/pralhad/AppData/Local/Programs/Python/Py  
Traceback (most recent call last):  
  File "c:\Users\pralhad\Desktop\ma\app.py", line 1, in <module>  
    with open("sample.txt", "r") as f: # Relative path pointing to the current dir  
FileNotFoundError: [Errno 2] No such file or directory: 'sample.txt'
```

- We get **FileNotFoundError**

Why Does This Happen?

- Python is still looking for sample.txt in the **same directory as** main.py,
- But the file has been moved to the data/ subfolder.

Fixing the Path

To fix this, we need to tell Python that the **file is now inside the data/ folder**.

Folder Structure:

```
repo/  
├── data/  
│   └── sample.txt  
└── main.py
```

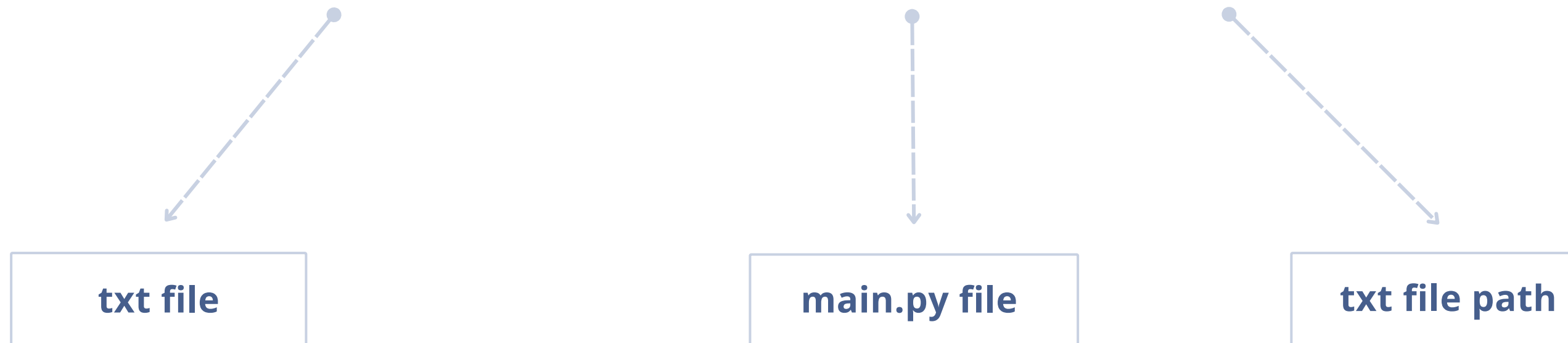
```
with open("data/sample.txt", "r") as f:  
    content = f.read()  
print(content)
```

Relative to CWD

Output:

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS  
● PS C:\Users\pralhad\Desktop\project> & C:/Users/pralhad/AppData/Local/Programs/Python/Python39-64/Scripts/python.exe .\main.py  
  This is a test line.  
○ PS C:\Users\pralhad\Desktop\project>
```

Relative Filepath = **CWD + Filepath**



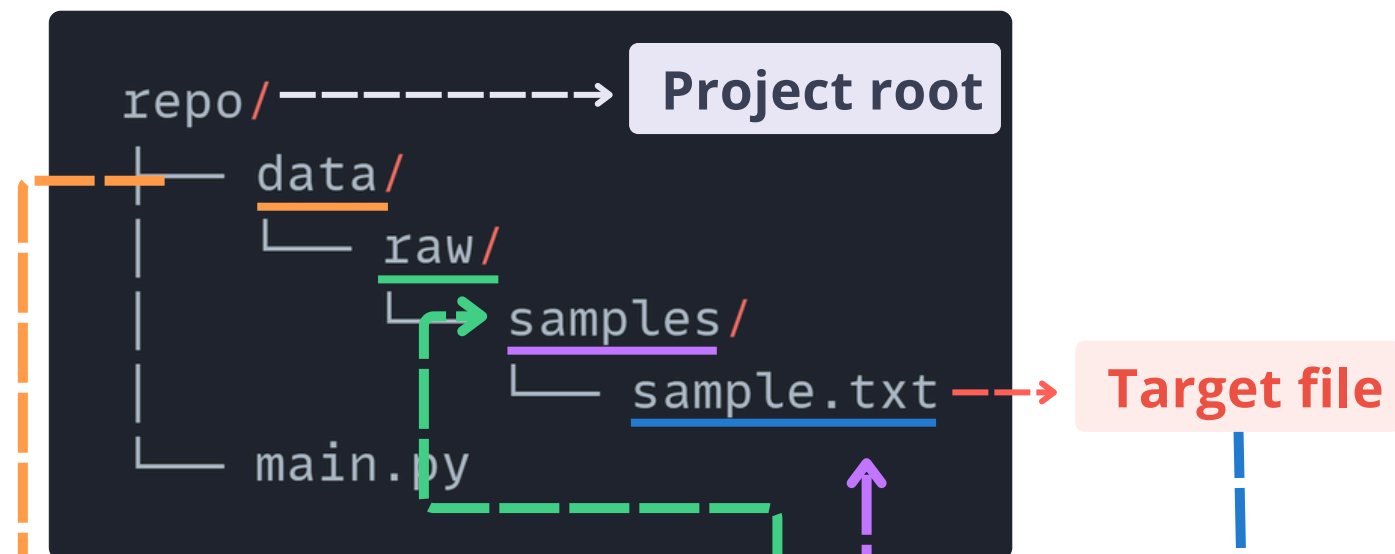
How Does Python Find the File?

- Python starts looking in the **current working directory (CWD)**—the folder where the script is run.
- From there, it follows the relative path you provide.
- **Example:**
 - **"data/sample.txt"** tells Python to look inside the **data** folder for **sample.txt**.

File in Multiple Subfolders

Now, let's see what happens if the file we need is moved deep inside multiple subfolders.

Folder Structure:



How Python finds the file:

- Python searches from root
- It follows the relative path like a trail:
 - *cwd (root)* → *data* → *raw* → *samples* → *sample.txt*

```
with open("data/raw/samples/sample.txt", "r") as f:
    content = f.read()

print(content)
```

A Large Project Structure

Now, let's move main.py into a new scripts/ folder and place sample.txt in a separate data/raw/

Folder Structure:

```
repo/  
├── data/  
│   └── input_data.txt  
├── scripts/  
│   └── process_data.py
```

File to read

Script

How Python finds the file:

- Python searches from root
- It follows the relative path like a trail:
 - *cwd (repo/) → scripts → data → input_data.txt*

scripts/process_data.py:

```
with open("data/input_data.txt", "r") as file:  
    content = file.read()  
  
print(content)
```

Incorrect relative path

Output:

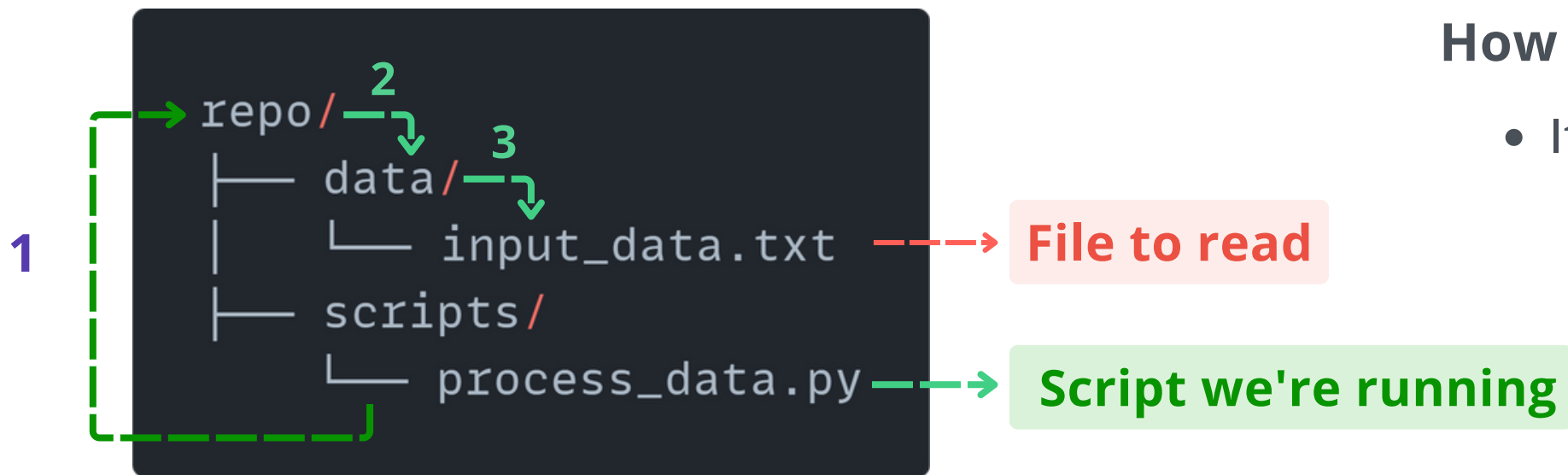
```
FileNotFoundError: [Errno 2] No such file or directory: 'data/input_data.txt'
```

- There is no **data/** folder inside **scripts/**,

To fix it, we need to tell Python to:

1. Go up one level from **scripts/**
2. Then into the **data/** folder

Folder Structure:

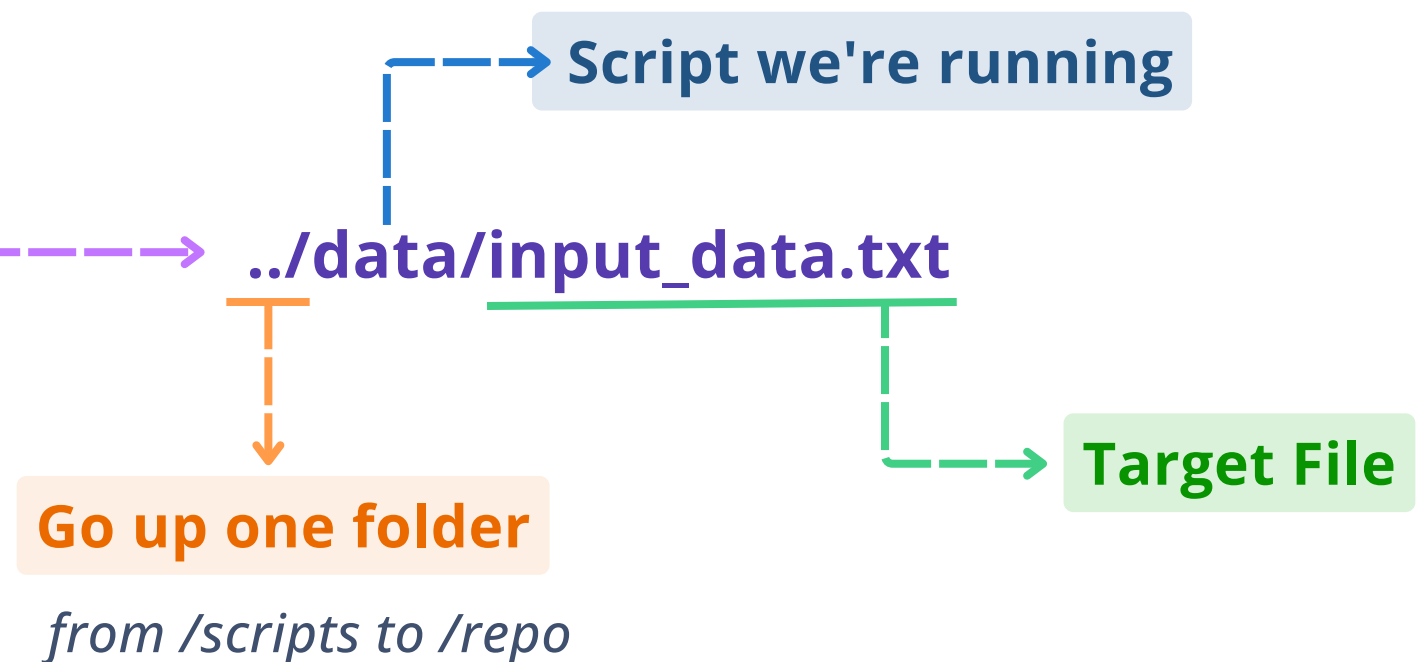


How Python finds the file:

- It follows the relative path like a trail:
 - *cwd (scripts/) → .. (go up to repo/) → data → input_data.txt*

process_data.py

```
with open("../data/input_data.txt", "r") as file:
    content = file.read()
print(content)
# Output : This is input data.
```



Key Points about Relative Path:

- It does **not** start from the root of the file system.
- It starts from current script folder.
- It uses **folders** relative to the script's location.
- It's **flexible** and **portable**, making it ideal for sharing code across different environments.

Relative Filepath = CWD (Script Directory) + Filepath



CP - Relatives Path

File in Same Directory

Folder Structure:

```
repo/  
├── sample.txt  
└── main.py
```

Content of sample.txt:

```
Hello, this is a test file.
```

Code in main.py:

```
with open("sample.txt", "r") as file:  
    content = file.read()  
print(content)  
  
# Output: Hello, this is a test file.
```

File in a Subfolder

Folder Structure:

```
repo/  
├── data/  
│   └── sample.txt  
└── main.py
```

Content of sample.txt:

```
Data stored in a subfolder.
```

Code in main.py:

```
with open("data/sample.txt", "r") as file:  
    content = file.read()  
print(content)  
  
# Output: Data stored in a subfolder.
```

File in Nested Subfolders

Folder Structure:

```
repo/  
├── data/  
│   └── raw/  
│       └── samples/  
│           └── sample.txt  
└── main.py
```

Content of sample.txt:

```
This is a deeply nested file.
```

Code in main.py:

```
with open("data/raw/samples/sample.txt", "r") as file:  
    content = file.read()  
print(content)  
  
# Output: This is a deeply nested file.
```

File in a Parallel Folder

Folder Structure:

```
repo/  
├── data/  
│   └── input_data.txt  
└── scripts/  
    └── process_data.py
```

Content of input_data.txt:

```
Parallel folder data content.
```

Code in process_data.py:

```
with open("../data/input_data.txt", "r") as file:  
    content = file.read()  
print(content)  
  
# Output: Parallel folder data content.
```


Non-Existent File Handling

Folder Structure:

```
repo/  
└─ main.py
```

Code in main.py:

```
try:  
    with open("nonexistent.txt", "r") as file:  
        content = file.read()  
        print(content)  
except FileNotFoundError:  
    print("File not found. Please check the path and try again.")
```

- This demonstrates how to handle errors when the specified file does not exist in the directory.
- The output will be:

```
File not found. Please check the path and try again.
```

Absolute Path

- What is an Absolute Path?
- How Python Finds a File Using an Absolute Path
- Reading a File Using an Absolute Path
- Load File from the Same Project (using Absolute Path)
- Load File from a Different Drive/Folder (Custom Location)
- Important Things to Note
- Hard Coded Paths
- Why Avoid Hard-Coded Paths?
- When to Use Relative Paths
- When to Use Absolute Paths
- Questions Time
- Answers

What is an Absolute Path?

- An **absolute path** is the full, exact location of a file or folder — starting from the **root** of computer's file system.
- Think of it like a **full home address**, starting from the country all the way to the **house number**.

Key Points About Absolute Paths

Method

- **Ignores current script folder**
- **Starts from root**

Purpose

- It **does not depend** on the current working directory.
- It provides the **complete address** to the file or folder.
- Windows: C:\ — macOS/Linux: /

Example of an Absolute Path:

- **On Windows:** `C:\Users\Dev\Documents\sample.txt`
- **On Linux/Mac:** `/home/user/Documents/sample.txt`

Reading a File Using an Absolute Path

Folder Structure:



Reading a File Using an Absolute Path

```
with open(r"C:\Users\Dev\Documents\sample.txt", "r") as f:
    content = f.read()
print(content)
# Output: This is a test line.
```

The diagram highlights the following elements:

- R String:** Points to the `r` prefix in the file path.
- Absolute text:** Points to the entire file path `"C:\Users\Dev\Documents\sample.txt"`.

Why Use r Before the Path?

In this line:

```
r"C:\Users\Dev\Documents\sample.txt"
```



- The **r** means it's a **raw string**.
- It tells Python not to process backslashes (\) as **escape characters**.

Without r, This Could Go Wrong:

If you wrote:

```
"C:\Users\Dev\Documents\sample.txt"
```

- Python might interpret:
 - **\n** → as a newline
 - **\t** → as a tab
- This can corrupt your path unintentionally.

What Happens When This Code Runs?

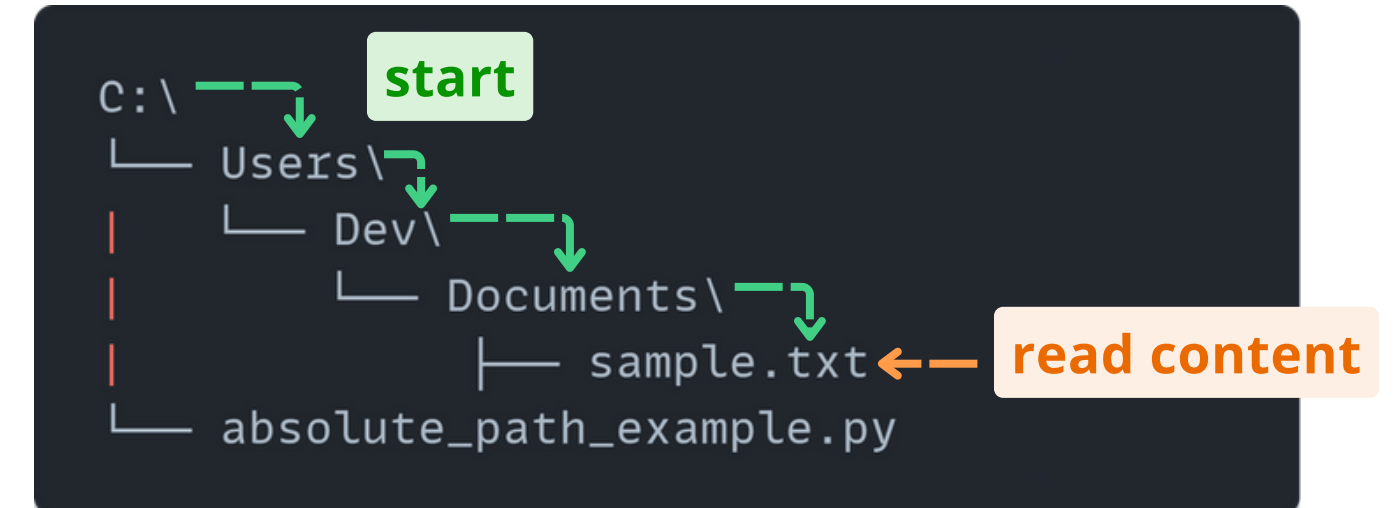
- Python starts from **C:**
- Then moves into **Users**, then **Dev**, then **Documents**
- It finds **sample.txt** and **reads** the **content**
- It **prints**:

```
This is a test line.
```

✓ Why It Works Anywhere

- It **doesn't** matter where the **python script** file is **located** and run from.
- **Python** finds the file because the entire path is **fully** written

Folder Structure:



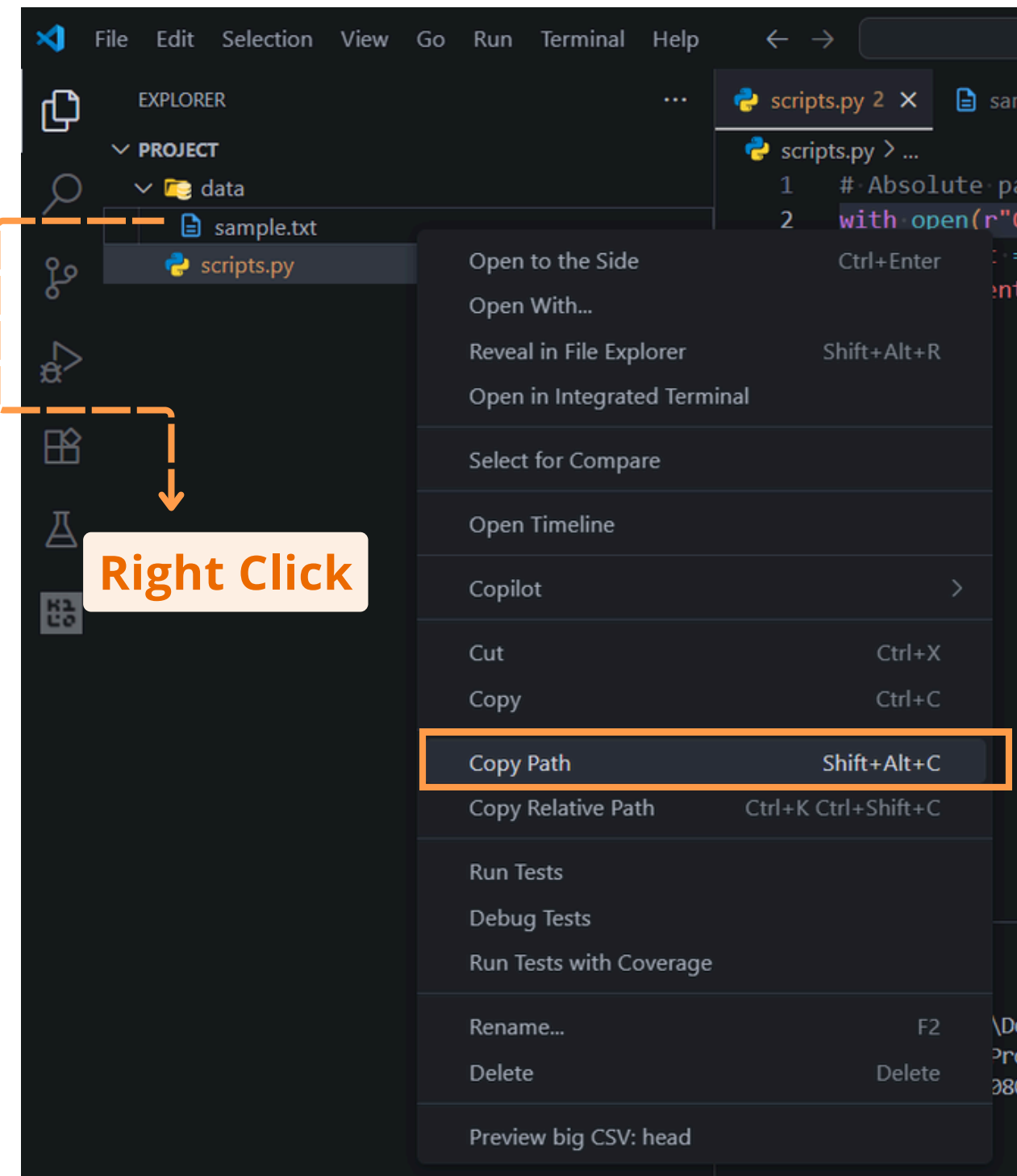
How Python Find the File?

When you give **Python** an **absolute path**, here's exactly what it does:

- **Starts from the root of the file system:**
 - **Windows** → C:\
 - **Linux/macOS** → /
- **Follows the exact folder structure you provide:**
 - One folder at a time, just like navigating through File Explorer or Finder.
- **Finds and opens the file**, assuming it exists and you have permission.

How to Copy the Full File Path

In Visual Studio Code (VS Code):



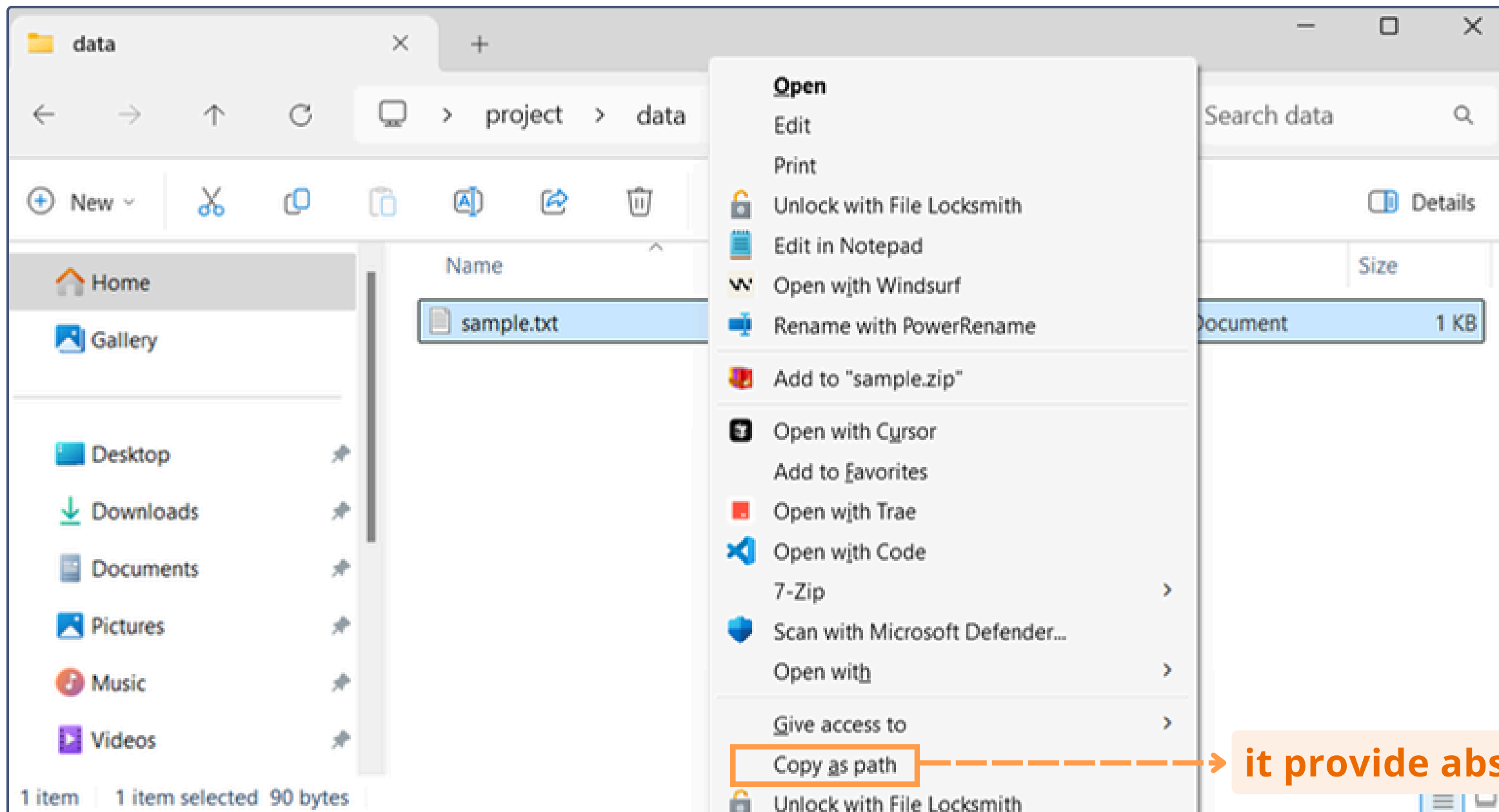
- **Right-click** on the file in the Explorer sidebar.
- Click "**Copy Path**".
- Now you can paste the path directly into your **Python script**.

Click to Copy as - Absolute Path

Copied Content

```
file_path = "C:\\Users\\YourName\\Documents\\data.txt"
```


In Windows File Explorer:



- Hold **Shift**, then **right-click** on the file.
- Choose "**Copy as Path**" from the menu.
- This gives you the **full file path as text**.

Output

```
file_path = "C:\\Users\\YourName\\Documents\\data.txt"
```

Load File from the Same Project (using Absolute Path)

In real-world projects — like **VFX** tools or **pipeline** utilities — your **script** and **data** files often live in different folder

Folder Structure:

```
repo/  
├── config/  
│   └── sample.txt  
└── scripts/  
    └── read_config.py
```

Contains configuration details

Python Code

```
with open(r"C:\Users\YourName\repo\config\sample.txt", "r") as f:  
    content = f.read()  
print(content)
```

Absolute Path

- Python starts at C:\ and follows the full path to the file.

Output

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS  
● PS C:\Users\pralhad\Desktop\project> & C:/Users/pralhad/App  
project_name = TestProject  
resolution = 1920x1080  
frame_rate = 24  
output_format = EXR  
○ PS C:\Users\pralhad\Desktop\project>
```

Load File from a Different Drive/Folder (Custom Location)

Sometimes your **data lives outside** your project folder — on **another drive** (like **D:**) or in a **permanent location** e.g., **shared department drive, external disk**

- This is where **absolute paths** are especially useful.

Scenario:

External Folder Structure

```
D:\
└─ Data\
    └─ external_file.txt
```

Content inside external_file.txt:

```
External data for analysis.
```

Script Location

```
project/
└─ scripts/
    └─ read_external.py
```

Python Script: read_external.py

```
with open(r"D:\Data\external_file.txt", "r") as f:
    content = f.read()
print(content)
```

read

Scenarios where Absolute Paths are used?

for files stored in **other places** on your **computer** or **network**.



- Shared Network Drive → Z:**Shared**\TeamData\config.txt
- External USB or Backup Drive → E:**Backup**\assets\model.fbx
- System-Wide Config Directory → C:**ProgramData**\AppName\settings.ini
- Cloud Sync Folder → D:**Dropbox**\VFX_References\hdris\sky.exr
- Team Collaboration Repository → \\renderfarm01\PipelineTools\global_settings.json

Why Absolute Paths Aren't Ideal

✓ Pros

- Always works on your computer
- **You give the full address:**
 - 🖱️ `C:/Users/John/Desktop/project/data/input_data.txt`
- Python just **follows the full path**.

✗ Cons

- **Breaks on other machines** your **teammate doesn't have**
 - `C:/Users/John/...`,
 - **the script crashes.**
- **Hard-coded to personal system path**
 - If we **move the project or** rename folders, **every path breaks.**

Hard Coded Paths

A **hard-coded** path is when you write the **full path** to a file directly in **script**.

Example:

```
file_path = "C:\\Users\\YourName\\Documents\\data.txt"
```

Hard Coded Paths

- It works **only** on **your** machine.
- It's **not calculated** dynamically and **doesn't adapt** to other **users** or **environments**.

Why Avoid Hard-Coded Paths?

1. Non-Portable Code

- The **script** **only** works on **your machine**.
- **Other users** may have different **usernames**, **drives**, or **folder setups**.
- They'll need to **manually change the code** just to make it work.

2. Hard to Maintain

- If we move the **file** or **restructure your project**:
 - You'll need to find and **update every hard-coded instance**.
 - This can be **error-prone** and **time-consuming** in **large scripts**.

3. Prone to Errors

- Most **common** symptom:

```
FileNotFoundError
```

- Why? The script points to a location that **doesn't exist** on the **current system**.

When to Use Relative Paths

These are best for **flexible, portable** code — especially in **collaborative** or **multi-system** projects.

Situation

- **Project Portability**
- **Collaboration**
- **Cleaner Code**
- **Nested Project Directories**

Why Relative Paths Work Well

- Your code should run on any system without modification
- Other users have the same project folder structure
- You avoid hardcoding system-specific paths
- Files live in folders relative to the script (e.g., `../data/`)

Example:

```
with open("../config/settings.txt", "r") as f:  
    ...
```

Relative Path

When to Use Absolute Paths

These are ideal for **fixed locations** outside your project or in **scripts** meant for **single-system** use.

Situation

- **Fixed File Location**
- **Single-System Use**
- **External Drives / Shared Folders**
- **No Need for Project Awareness**

Why Absolute Paths Make Sense

- File lives in a constant place (e.g., **C:/Logs/app.log**)
- You're writing tools for your own system
- File is on a different drive (**D:/Data/...**)
- Script doesn't depend on folder structure

Example:

```
with open(r"C:\ProgramData\myapp\config.ini", "r") as f:  
    ...
```

absolute path

Summary Table

Use Case	Relative Path ../data/file.txt	Absolute Path C:/data/file.txt
• Portable scripts	✓	✗
• Shared team projects	✓	✗
• Access outside project folder	✗	✓
• System-level files or logs	✗	✓
• Cleaner, adaptable code	✓	✗

Questions Time

- What is the **difference between Absolute and Relative Paths**?
- When **creating and loading a text file** in Python, are you using a **Relative Path or an Absolute Path**?
- How can you **identify** whether a **given path** is **Relative or Absolute**?

Answers

1. What is the difference between Absolute and Relative Paths?

- **Absolute Path:**

- A full path that starts from the **root** of the file system (like **C:/Users/Name/Documents/file.txt**).
- It shows exactly where the **file** is on **your computer**.

- **Relative Path:**

- A path that starts from the **location** of your **Python script** (like **../data/file.txt**).
- It depends on **where** the **script** is and how folders are **organized** in **your project**.

2. When creating and loading a text file in Python, are you using a Relative Path or an Absolute Path?

- It depends on **what path** you write in **your code**:

- If the path starts with something like **C:/**, **D:/**, or **/home/**, you're using an **absolute path**.
- If it starts with a folder name like **data/** or **../**, you're using a **relative path**.

3. How can you identify whether a given path is Relative or Absolute?

- **A path is absolute if:**
 - It starts from the **root** of **the file system**
 - (e.g., **C:**, **D:**, or / on **Linux/Mac**)
- **A path is relative if:**
 - It starts from the **current folder** or **uses ..** to go **up a level**
 - (e.g., **data/sample.txt**, **../config/settings.json**)

CP- Absolute Path

Reading a File in a Subdirectory

Folder Structure:

```
C:\
├── project\
│   └── subdir\
│       └── data.txt
```



Code:

```
with open(r"C:\project\subdir\data.txt", "r") as file:
    content = file.read()
print(content)

# Output will show the file's content
```

Storing Logs in a Specific Folder

Folder Structure:

```
C:\
├── Logs\
│   └── my_app.log
```



Code:

```
with open(r"C:\Logs\my_app.log", "w") as log_file:
    log_file.write("Application started successfully.\n")
print("Log written successfully.")
```

Accessing a File on a Network Drive

Folder Structure:

```
Z:\
├── Shared\
│   └── Projects\
│       └── report.pdf
```



Code:

```
with open(r"Z:\Shared\Projects\report.pdf", "rb") as file:
    content = file.read(100)
print("First 100 bytes of the file:", content)
```

Storing Logs in a Specific Folder

Folder Structure:

```
D:\
├── Data\
│   └── analysis\
│       └── results.csv
```



Code:

```
with open(r"D:\Data\analysis\results.csv", "r") as file:
    content = file.read()
print(content)

# Output: Displays the content of results.csv
```

Demonstrates how to access a file located on a different **drive (D:)** using an absolute path.

Thank You

